ENGINEERING DESIGN MANUAL

HD44780 LCD Driver Subsystem — Firmware Design Specification

Target MCU: STM32F407VG

Project: EV Charger Firmware (IEC 61851-1, Mode 3 AC)

LCD Support: 16x2 and 20x4 HD44780-Compatible Displays, 4-bit Parallel Interface

Document Classification: Internal Engineering Reference

Document No: FW-LCD-EVSE-0001

Revision: 1.0

Date: August 2026

Revision History

Rev	Date	Author	Description
0.1	2026-07-28	Firmware Team	Initial draft — BSP and driver skeleton review
0.5	2026-08-01	Firmware Team	Added architecture chapters, HD44780 internals
1.0	2026-08-04	Firmware Team	First complete internal release for EV charger LCD subsystem

Approval

Role	Name	Signature	Date
Firmware Architect			
Hardware Lead			
QA / Verification			

Table of Contents

List of Figures

List of Tables

Chapter 1 — Introduction

This document specifies the design of the LCD driver subsystem used by the EV charger firmware running on an STM32F407VG microcontroller. The subsystem drives an HD44780-compatible character LCD (16x2 or 20x4) over a 4-bit parallel interface. It is written as an internal engineering reference: every design decision is justified, every layer's responsibility is bounded explicitly, and every public function is documented to register level where relevant.

1.1 Why a Dedicated LCD Driver Is Needed

A charger's main.c is responsible for orchestrating system startup — clock configuration, peripheral initialization, and handing control to the application. It is not the right place to encode HD44780 protocol timing, nibble ordering, or DDRAM addressing. If LCD commands are written directly in main.c or inside the charger state machine, three problems appear immediately:

- Protocol details leak into business logic, making the state machine harder to read and verify against IEC 61851-1 requirements.
- Any change to the physical LCD (16x2 → 20x4, or parallel → I2C backpack) forces changes throughout the application instead of inside one isolated module.
- Unit testing becomes impossible without real hardware, because display logic and communication logic are inseparable.

1.2 Why Not Call HAL Directly From the Application

HAL_GPIO_WritePin() is a hardware primitive. Calling it from application code couples the charger's UI logic to specific GPIO ports and pins. A driver layer converts hardware primitives into a meaningful vocabulary — LCD_WriteString(), LCD_SetCursor() — that the application can use without knowledge of GPIO, timing, or the HD44780 instruction set.

1.3 Driver Abstraction and Hardware Abstraction

Two abstraction boundaries exist in this design, and they must not be merged:

- Hardware Abstraction (BSP): isolates which physical pins and ports implement RS, EN, and D4–D7. The BSP does not know what a 'command' or a 'character' is.
- Protocol/Driver Abstraction (LCD Driver): implements the HD44780 4-bit protocol — nibble splitting, timing, command encoding. The driver does not know which GPIO port RS is wired to.

> **NOTE: Two boundaries, two reasons to change**
>
> A class should have one reason to change. The BSP changes when the PCB changes. The driver changes when the LCD protocol requirements change. Merging them means a PCB revision risks breaking protocol-correct code, and vice versa.

1.4 Encapsulation, Maintainability, Portability

Functions such as LCD_SendNibble(), LCD_SendByte(), and the row-address lookup table are declared static in driver_lcd.c. They are implementation details of the driver and are intentionally invisible outside the translation unit. This is encapsulation in C: visibility is controlled by storage class and file scope rather than by language-level access modifiers.

Portability follows directly from the layering: to move this firmware to an STM32G4-based board revision, only the BSP implementation changes (different GPIO_TypeDef pointers, possibly a different HAL). driver_lcd.c is untouched because it never references a GPIO port or pin directly.

1.5 Scalability and Reusability

Because the driver is parameterized by LCD_ROWS and LCD_COLUMNS (Chapter 16) rather than hard-coded for one geometry, the identical driver source supports both 16x2 and 20x4 displays used across different charger hardware variants. The same BSP/driver pair can also be reused on any future STM32 product that uses a parallel HD44780 LCD, provided the BSP is re-implemented for the new pinout.

1.6 Bad Architecture vs Professional Architecture

Aspect	Bad Architecture (flat)	Professional Architecture (layered)
LCD commands	Issued directly in main.c or ISR	Issued only via LCD_* public API
GPIO access	Scattered across application files	Confined to BSP layer only
Portability	Requires rewriting application code	Requires only a new BSP
Testability	Untestable without full hardware	Driver logic testable with mocked BSP
Change impact	One display change ripples everywhere	Change is contained to one layer
Readability	Protocol and business logic interleaved	Each file has one responsibility

COMMON MISTAKE: Calling HAL_GPIO_WritePin() from the charger state machine

This is the most common shortcut taken under schedule pressure. It works in the short term and becomes very expensive to unwind once the state machine has grown to handle IEC 61851 states A–F, fault handling, and RFID/UI events.

Chapter 2 — Complete Firmware Architecture

The subsystem is organized into six layers. Each layer communicates only with its immediate neighbor, never skipping a level. This section defines the responsibilities, inputs, outputs, dependencies, and design rationale for every layer.

Figure 2.1 — Complete Firmware Layering (Application → Hardware)

2.1 Application Layer

Attribute	Description
Responsibility	Charger business logic: IEC 61851 state machine, RFID authorization, fault handling
Inputs	CP state transitions, RFID events, fault flags, timers
Outputs	Requests to the UI layer such as 'show charging status'
Dependencies	UI layer only
Must never call	LCD_*, BSP_*, or HAL_* directly

2.2 UI Layer

Attribute	Description
Responsibility	Translates application state into human-readable screens/lines
Inputs	Application-level events (e.g. STATE_CHARGING, FAULT_GFCI)
Outputs	Calls to LCD_SetCursor() / LCD_WriteString()
Dependencies	LCD Driver public API only
Advantage	Screen layout can change without touching the state machine

2.3 LCD Driver Layer

Attribute	Description
Responsibility	Implements the HD44780 4-bit protocol correctly and completely
Inputs	Row/column, characters, strings from the UI layer
Outputs	Calls to BSP_LCD_* functions
Dependencies	BSP layer only
Ownership	Owns command macros, DDRAM address table, init sequence

2.4 BSP Layer

Attribute	Description
Responsibility	Maps abstract pin roles (RS, EN, D4-D7) to physical GPIO
Inputs	Logical requests: set RS, write nibble, pulse enable, delay

Attribute	Description
Outputs	HAL_GPIO_WritePin() calls, HAL_Delay() / cycle-counter delay
Dependencies	STM32 HAL only
Constraint	Only layer permitted to reference GPIOx_Pin / GPIOx_GPIO_Port macros

2.5 STM32 HAL Layer

Provided by ST. Wraps register-level GPIO/RCC access behind HAL_GPIO_WritePin(), HAL_GPIO_Init(), and similar functions. Discussed in depth in Chapter 14.

2.6 GPIO / Hardware

Physical GPIOD pins in this design (per the generated main.c) driving LCD_RS, LCD_EN, LCD_D4–LCD_D7, and BUZZER. The HD44780 controller inside the LCD module receives these transitions and updates DDRAM/CGRAM and the display accordingly.

2.7 Design Decisions Summary

- Strict downward-only calling: a layer may call the layer directly below it, never above.
- No layer skipping: the driver never calls HAL directly, even though it would save one function call, because that would leak GPIO knowledge into the driver.
- Static linkage for internal helpers to prevent accidental cross-module calls.
- Blocking, polled communication chosen over interrupt-driven or DMA-based LCD updates, because LCD refresh is low-frequency and not timing-critical relative to the charger control loop.

> **BEST PRACTICE: Downward-only dependency rule**
>
> If you find yourself importing bsp_lcd.h into the application layer 'just this once', that is a signal the layering has been violated and should be fixed before it becomes a habit.

Chapter 3 — Folder Structure

The repository is organized so that a new team member can locate any file by its responsibility, not by guessing. The structure mirrors the architecture layers described in Chapter 2.

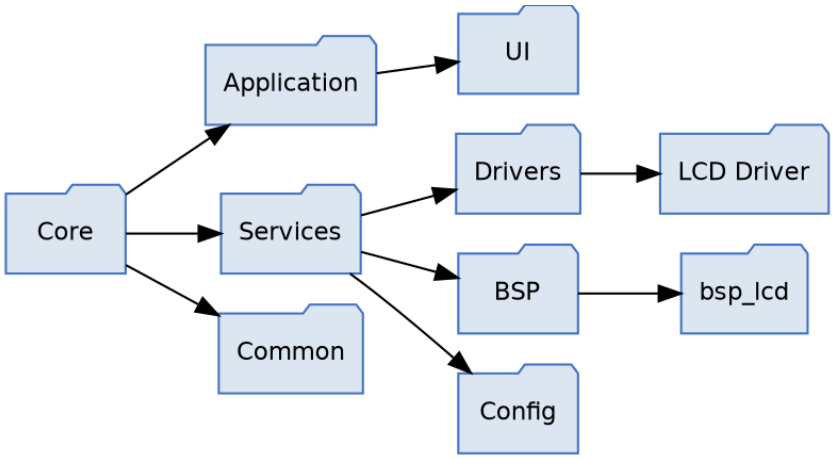

Figure 3.1 — Recommended Folder Structure

Core/

Vendor-generated startup code, linker scripts, and CMSIS headers produced by STM32CubeIDE. Nothing in this folder is hand-edited except inside USER CODE BEGIN/END guards.

Application/

Contains App_Init()/App_Run() and the charger state machine. Depends only on Services/UI. No GPIO or HAL includes are permitted here.

Services/UI/

Screen composition logic: which line shows which status text, scrolling behavior, menu handling. Only file allowed to call the public LCD driver API for display purposes.

Services/Drivers/

Protocol-level drivers: driver_lcd.c/h lives here, alongside any future drivers (e.g. driver_rfid.c). Each driver depends only on its corresponding BSP module and on HAL types indirectly via main.h.

Services/BSP/

Board Support Package: bsp_lcd.c/h. This is the only folder permitted to contain GPIOx_Pin / GPIOx_GPIO_Port literals for the LCD subsystem.

Services/Config/

Compile-time configuration such as LCD_ROWS / LCD_COLUMNS, feature flags, and board-revision selection macros.

Common/

Shared utility code with no hardware dependency: string formatting helpers, ring buffers, generic macros.

> **COMMON MISTAKE: Placing bsp_lcd.c under Application/**

> This is a frequent misfile that reintroduces the exact coupling the architecture is designed to prevent — GPIO literals become reachable from application-level includes.

Chapter 4 — HD44780 Internal Architecture

The HD44780 is not a passive shift register; it is a small microcontroller-like peripheral with its own registers, memory, and internal state machine. Understanding its internals explains why the driver must send bytes in a specific order and wait specific amounts of time.

4.1 Instruction Register (IR) and Data Register (DR)

The HD44780 exposes two 8-bit registers to the host. RS=0 selects the Instruction Register: bytes written here are interpreted as commands (clear display, set DDRAM address, function set, etc.). RS=1 selects the Data Register: bytes written here are treated as character codes to be written at the current DDRAM or CGRAM address, and the internal address counter auto-increments or decrements per the entry-mode setting.

4.2 DDRAM — Display Data RAM

DDRAM holds the character codes currently mapped to visible display positions. Each row of the display corresponds to a fixed base address in DDRAM (Chapter 12 covers the exact map for 16x2 and 20x4 geometries). Writing a character code to DDRAM at the current address makes that character appear on the physical display.

4.3 CGRAM — Character Generator RAM

CGRAM allows up to 8 custom 5x8 pixel characters (character codes 0x00–0x07) to be defined by the application. Selecting CGRAM address mode uses command 0x40 (LCD_CMD_SET_CGRAM_ADDR); subsequent data writes program pixel rows rather than DDRAM character codes.

4.4 Character Generator ROM (CGROM)

The fixed built-in font table. Character codes 0x20–0x7F map to standard ASCII glyphs, so writing the ASCII value of a printable character is sufficient for normal text — this is why LCD_WriteChar() can simply cast a C char to uint8_t and send it as-is.

4.5 Busy Flag and Address Counter

A real HD44780 exposes a busy flag on D7 when RW=0 is asserted during a read, allowing the host to poll for readiness instead of using fixed delays. This design ties RW permanently LOW (write-only), so the busy flag is never read; all timing is instead guaranteed using fixed worst-case delays per the datasheet. This is a deliberate simplification discussed further in Chapter 5.

The Address Counter (AC) tracks the current DDRAM or CGRAM address and increments/decrements automatically after each data write, per the entry-mode command (0x06 = increment).

4.6 Internal State Machine

Figure 4.1 — HD44780 Internal State Transitions

On power-up, the controller's internal state is undefined until either the internal power-on reset circuit completes (not guaranteed reliable across all clone/no-name modules) or the host performs the software reset sequence: three 0x03 nibbles followed by 0x02. This is why LCD_Init() always performs the explicit software sequence rather than trusting the internal power-on reset — it makes initialization deterministic across the wide variety of HD44780-compatible controllers found in low-cost LCD modules.

4.7 How the LCD Processes Every Received Command

Step	Internal Action
1	RS and RW sampled to select target register (IR or DR) and direction
2	Upper nibble latched into internal shift register on EN falling edge
3	Lower nibble latched, combined with upper nibble into full 8-bit value
4	Instruction decoder interprets the byte (if RS=0) or writes it to DDRAM/CGRAM at AC (if RS=1)
5	Address Counter updated per entry mode
6	Busy flag internally asserted for the instruction's execution time (Chapter 11)

Chapter 5 — Hardware Interface

This chapter documents every physical pin on a standard 16-pin HD44780 header and the electrical reasoning behind the choices made in this design.

Pin	Name	Function	Used in this design
1	VSS	Ground	Yes — GND
2	VDD	Logic supply, typically 5V or 3.3V module-dependent	Yes
3	VEE / V0	Contrast adjustment voltage	Yes — via potentiometer, not GPIO-controlled
4	RS	Register Select (0=Instruction, 1=Data)	Yes — LCD_RS_Pin
5	RW	Read/Write select	Tied LOW (write-only), not driven by GPIO
6	EN	Enable — latches data on falling edge	Yes — LCD_EN_Pin
7-10	D0-D3	Lower data nibble	Not used (4-bit mode)
11-14	D4-D7	Upper data nibble / 4-bit data bus	Yes — LCD_D4..LCD_D7_Pin
15	A (LED+)	Backlight anode	Board-dependent, not covered here
16	K (LED-)	Backlight cathode	Board-dependent, not covered here

5.1 Why RW Is Tied LOW

Tying RW permanently to ground fixes the interface in write-only mode, at the cost of never being able to poll the busy flag. This trade-off is intentional: it frees one GPIO pin (useful on pin-constrained packages) and simplifies the driver, at the cost of using fixed conservative delays instead of polling. For a UI-refresh-rate LCD, this cost is negligible.

> **NOTE: Trade-off, not a shortcut**
>
> Tying RW low is a legitimate, common design pattern in embedded LCD interfacing — not a simplification taken for lack of time. It is documented here so future engineers do not 'fix' it by wiring RW to a GPIO without understanding why it was omitted.

5.2 Why Only D4–D7 Are Used (4-bit Mode)

4-bit mode transmits each byte as two 4-bit nibbles instead of one 8-bit transfer, at the cost of doubling the number of enable pulses per byte. In exchange, it frees four GPIO pins (D0–D3), which matters on the STM32F407VG where GPIOD is shared with other functions (BUZZER in this design). This is standard practice for embedded LCD interfacing and is reflected in Function Set command 0x28 (4-bit, 2-line).

5.3 Electrical Behavior

- All LCD control/data lines are configured GPIO_MODE_OUTPUT_PP (push-pull) at GPIO_SPEED_FREQ_LOW, matching main.c's MX_GPIO_Init().
- No pull-up/pull-down is required on RS/EN/D4-D7 since they are always actively driven by the MCU.
- VEE/contrast is set once via a physical potentiometer and is out of firmware's control.

5.4 Timing Overview

Full electrical timing (setup time, hold time, enable pulse width) is covered with a waveform diagram in Chapter 15. In summary: RS/data must be stable before EN rises, EN must remain high for a minimum pulse width, and data is latched on the EN falling edge.

Chapter 6 — BSP Layer

This chapter documents every BSP function individually to the level of detail required for a new engineer to modify or port the BSP without introducing regressions.

6.1 BSP API Summary

Function	Purpose
BSP_LCD_SetRS(state)	Selects Instruction (0) or Data (1) register
BSP_LCD_WriteNibble(nibble)	Drives D4-D7 with the low 4 bits of nibble
BSP_LCD_PulseEnable(void)	Generates the EN high-then-low pulse that latches data
BSP_LCD_DelayUs(us)	Provides a delay used for LCD timing compliance

6.2 BSP_LCD_SetRS()

```
void BSP_LCD_SetRS(uint8_t state)
{
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin,
                      state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

Attribute	Detail
Purpose	Select Instruction Register (state=0) or Data Register (state=1)
Prototype	void BSP_LCD_SetRS(uint8_t state)
Arguments	state — 0 selects command mode, non-zero selects data mode
Return value	None
Caller	LCD_SendByte() exclusively
Callee	HAL_GPIO_WritePin()
Hardware behavior	Drives LCD_RS_Pin HIGH or LOW immediately; no delay is required here because the delay is inserted later, before EN is pulsed

NOTE: Line-by-line

The ternary expression state ? GPIO_PIN_SET : GPIO_PIN_RESET exists because HAL's GPIO_PinState enum is not numerically identical to a plain boolean in all HAL versions; the ternary makes the mapping explicit and avoids relying on implicit integer-to-enum conversion.

6.3 BSP_LCD_WriteNibble()

```
void BSP_LCD_WriteNibble(uint8_t nibble)
{
    HAL_GPIO_WritePin(LCD_D4_GPIO_Port, LCD_D4_Pin,
                      (nibble & 0x01U) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D5_GPIO_Port, LCD_D5_Pin,
```

```
                      (nibble & 0x02U) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D6_GPIO_Port, LCD_D6_Pin,
                      (nibble & 0x04U) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D7_GPIO_Port, LCD_D7_Pin,
                      (nibble & 0x08U) ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

Each bit of the incoming nibble is isolated with a bitmask (0x01, 0x02, 0x04, 0x08) and mapped to its corresponding physical pin: bit 0 → D4, bit 1 → D5, bit 2 → D6, bit 3 → D7. This ordering matches the HD44780 4-bit interface convention where D4 is the least significant bit of the transmitted nibble.

WARNING: Port/pin consistency

Each HAL_GPIO_WritePin() call must reference the GPIO_Port that actually corresponds to its own pin macro (e.g. LCD_D6_GPIO_Port with LCD_D6_Pin). Referencing the wrong port compiles without error and may still function correctly if all four pins happen to share one port in the current PCB revision — but it silently breaks the moment any one pin is moved to a different port in a future hardware revision, since GPIO_TypeDef pointers for different ports are distinct.

6.4 BSP_LCD_PulseEnable()

```
void BSP_LCD_PulseEnable(void)
{
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    BSP_LCD_DelayUs(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
}
```

Attribute	Detail
Purpose	Generate the EN pulse that latches the currently-held nibble into the HD44780
Timing	EN held HIGH for approximately 1us (module-dependent minimum is typically 450ns), then driven LOW
Hardware behavior	The falling edge of EN is the actual latch event; the HIGH duration only needs to satisfy the minimum pulse-width spec
Common mistake	Omitting the delay entirely — on a fast Cortex-M4 at 168MHz, back-to-back GPIO writes with no delay can violate the minimum EN pulse width on some LCD modules

6.5 BSP_LCD_DelayUs()

```
void BSP_LCD_DelayUs(uint8_t us)
{
    HAL_Delay(1);
}
```

In the current implementation this function ignores its us argument and always blocks for approximately 1 millisecond via HAL_Delay(). This is functionally safe — 1ms always exceeds every microsecond-level requirement in the HD44780 timing spec — but it is not truly microsecond-accurate and makes LCD_Init() and every character write slower than the datasheet requires. This is documented as a known characteristic of the current implementation and is intentionally out of scope for this revision.

COMMON MISTAKE: Assuming BSP_LCD_DelayUs(150) waits 150us

It currently waits ~1ms regardless of the argument. Any timing-sensitive code that depends on true microsecond resolution must not rely on this function as currently implemented.

6.6 HAL_GPIO_WritePin() Internals

HAL_GPIO_WritePin() does not perform a read-modify-write on the GPIO output data register (ODR). Instead it writes to the port's BSRR (Bit Set/Reset Register):

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin,
                       GPIO_PinState PinState)
{
  if (PinState != GPIO_PIN_RESET)
    GPIOx->BSRR = (uint32_t)GPIO_Pin;         // set bits, lower half
  else
    GPIOx->BSRR = (uint32_t)GPIO_Pin << 16U;  // reset bits, upper half
}
```

Writing to BSRR is a single atomic write instruction: setting a bit in the lower 16 bits sets the corresponding ODR bit, and setting a bit in the upper 16 bits clears it. Because this is a single write (not read-modify-write on ODR directly), it is safe even if an interrupt modifies other pins on the same port between the read and write — a hazard that a naive 'ODR |= mask' implementation would not avoid.

Register	Role
ODR	Holds the current output level of each pin on the port (readable)
BSRR	Write-only; bit N sets pin N, bit N+16 resets pin N, atomically
Pin mask	GPIO_Pin is a bitmask like 0x0010 for pin 4, not a pin number

Chapter 7 — LCD Driver Layer

driver_lcd.c implements the HD44780 4-bit protocol on top of the four BSP primitives. It exposes a small, deliberately minimal public API in driver_lcd.h, and hides every protocol detail behind static functions.

7.1 Public vs Private APIs

Category	Functions	Visibility
Public	LCD_Init, LCD_Clear, LCD_Home, LCD_SetCursor, LCD_WriteChar, LCD_WriteString, LCD_DisplayOn, LCD_DisplayOff	Declared in driver_lcd.h, external linkage
Private	LCD_SendNibble, LCD_SendByte, LCD_SendCommand, LCD_SendData	static, file-scope only in driver_lcd.c

7.2 Encapsulation and Visibility

The static keyword gives these four helper functions internal linkage: the symbol does not appear in the object file's exported symbol table, so no other translation unit can call LCD_SendNibble() even if it declared a matching prototype. This is C's mechanism for information hiding, equivalent in intent to a 'private' method in an object-oriented language.

> **BEST PRACTICE: Prototype block at the top of the .c file**
>
> driver_lcd.c declares all four static function prototypes in a single block before any definitions. This lets the functions be defined in a natural top-down reading order (LCD_Init() near functions that use it) while the compiler still sees forward declarations for functions used before they are defined textually.

7.3 Driver-Level Architecture Diagram

Within the driver, calls flow strictly downward from public API to private helper to BSP:

```
Public API (LCD_WriteString, LCD_SetCursor, ...)
        |
        v
Private helpers (LCD_SendData/Command -> LCD_SendByte -> LCD_SendNibble)
        |
        v
BSP_LCD_* functions
```

Chapter 8 — Private Driver APIs

8.1 LCD_SendNibble()

```
static void LCD_SendNibble(uint8_t nibble)
{
    BSP_LCD_WriteNibble(nibble);
    BSP_LCD_PulseEnable();
}
```

Attribute	Detail
Purpose	Transfer exactly 4 bits to the LCD and latch them with one EN pulse
Parameters	nibble — low 4 bits significant, upper 4 bits ignored by BSP_LCD_WriteNibble()
Return value	None
Execution	Two sequential calls: place bits on D4-D7, then pulse EN to latch

Example: nibble = 0xA (binary 1010). D7=1, D6=0, D5=1, D4=0. This function has no concept of 'command' or 'character' — that distinction is entirely the responsibility of RS, which is set earlier by LCD_SendByte() and not touched here.

8.2 LCD_SendByte()

```
static void LCD_SendByte(uint8_t byte, uint8_t rs)
{
    BSP_LCD_SetRS(rs);
    LCD_SendNibble((byte >> 4) & 0x0F);   // upper nibble first
    LCD_SendNibble(byte & 0x0F);          // lower nibble second
}
```

Bit shifting explanation: (byte >> 4) & 0x0F right-shifts the byte 4 positions, moving the upper nibble into the lower 4 bits, then masks off anything above bit 3 (defensive masking; shifting an 8-bit value right by 4 already leaves the top 4 bits as 0 for unsigned types, but the mask keeps intent explicit and is a common defensive-coding convention when byte promotion to int is involved). byte & 0x0F isolates the lower nibble directly.

Worked example — sending ASCII 'A' (0x41)

Step	Value	Binary
byte	0x41	0100 0001
upper nibble = (byte >> 4) & 0x0F	0x04	0000 0100
lower nibble = byte & 0x0F	0x01	0000 0001

The HD44780 4-bit interface convention requires the upper nibble first — this is fixed by the controller's internal state machine once 4-bit mode is entered, not a driver choice.

> **COMMON MISTAKE: Sending lower nibble first**
>
> Reversing the nibble order produces garbage characters or apparently random behavior after initialization, because the controller interprets the two 4-bit halves in the wrong order for every subsequent byte.

8.3 LCD_SendCommand()

```
static void LCD_SendCommand(uint8_t command)
{
```

```
    LCD_SendByte(command, 0U);
}
```

Thin wrapper fixing RS=0. Exists so call sites (LCD_Clear(), LCD_Init(), etc.) read as intent ('send a command') rather than repeating the RS argument everywhere.

8.4 LCD_SendData()

```
static void LCD_SendData(uint8_t data)
{
    LCD_SendByte(data, 1U);
}
```

Symmetric wrapper fixing RS=1, used by LCD_WriteChar().

8.5 Private API Call Graph

```
LCD_SendCommand(cmd)  --> LCD_SendByte(cmd, 0) --+
                                                 |--> LCD_SendNibble(high)
LCD_SendData(data)    --> LCD_SendByte(data,1) --+--> LCD_SendNibble(low)
```

Chapter 9 — Public Driver APIs

9.1 LCD_Init()

Purpose: bring the HD44780 from an unknown power-up state to a known, configured, ready-to-write state. Prototype: void LCD_Init(void). Detailed in full in Chapter 10 (Initialization Sequence).

9.2 LCD_Clear()

```
void LCD_Clear(void)
{
    LCD_SendCommand(LCD_CMD_CLEAR_DISPLAY);
    HAL_Delay(2);
}
```

Sends command 0x01 and blocks for 2ms because Clear Display is documented as the slowest HD44780 instruction (~1.52ms typical, up to ~1.64ms at some supply voltages/clock rates); 2ms gives worst-case margin using a millisecond-granularity delay source.

9.3 LCD_Home()

```
void LCD_Home(void)
{
    LCD_SendCommand(LCD_CMD_RETURN_HOME);
    HAL_Delay(2);
}
```

Returns DDRAM address to 0x00 and resets any display shift, without clearing DDRAM contents. Also a slow instruction requiring the same 2ms margin.

9.4 LCD_DisplayOn() / LCD_DisplayOff()

```
void LCD_DisplayOn(void)  { LCD_SendCommand(LCD_CMD_DISPLAY_ON);  }
void LCD_DisplayOff(void) { LCD_SendCommand(LCD_CMD_DISPLAY_OFF); }
```

Toggle display visibility (0x0C / 0x08) without altering DDRAM contents — useful for blink-free power-saving or intentional blanking without a Clear Display's 2ms penalty.

9.5 LCD_SetCursor()

```
static const uint8_t LCD_RowAddress[] = {
    LCD_ROW_0_ADDRESS, LCD_ROW_1_ADDRESS,
    LCD_ROW_2_ADDRESS, LCD_ROW_3_ADDRESS
};

void LCD_SetCursor(uint8_t row, uint8_t col)
{
    uint8_t address;
    if (row >= LCD_ROWS) return;
    if (col >= LCD_COLUMNS) return;
    address = LCD_RowAddress[row] + col;
    LCD_SendCommand(LCD_CMD_SET_DDRAM_ADDR | address);
}
```

Bounds-checks row against LCD_ROWS and col against LCD_COLUMNS before touching the lookup table — this prevents both an out-of-bounds array read on LCD_RowAddress[] and sending a nonsensical DDRAM address to the controller. The final command ORs the base address with 0x80

(LCD_CMD_SET_DDRAM_ADDR), since the Set DDRAM Address instruction is encoded as 1AAAAAAA in the HD44780 instruction set.

> **NOTE: Silent return on invalid input**
> LCD_SetCursor() silently ignores out-of-range row/column instead of asserting or returning an error code. This is appropriate for a UI-facing API in a system without a debugger attached at runtime, but callers should be validated at the UI layer if incorrect coordinates would indicate a logic bug worth catching in debug builds.

9.6 LCD_WriteChar()

```
void LCD_WriteChar(char ch)
{
    LCD_SendData((uint8_t)ch);
}
```

The explicit (uint8_t) cast documents that char's signedness is implementation-defined in C, and the driver intends the raw 8-bit pattern to be sent regardless of whether char is signed or unsigned on the target toolchain.

9.7 LCD_WriteString()

```
void LCD_WriteString(const char *str)
{
    if (str == NULL) return;
    while (*str != '\0')
    {
        LCD_WriteChar(*str);
        str++;
    }
}
```

NULL-checked before dereferencing — a defensive habit that matters in embedded code where a misconfigured string pointer can otherwise cause a hard fault. The loop is a standard NUL-terminated string walk with no length limit; the caller is responsible for ensuring the string, combined with the current cursor position, does not exceed LCD_COLUMNS (LCD_WriteString() does not wrap or clip).

9.8 Complete Call Flow: LCD_WriteString() to Hardware

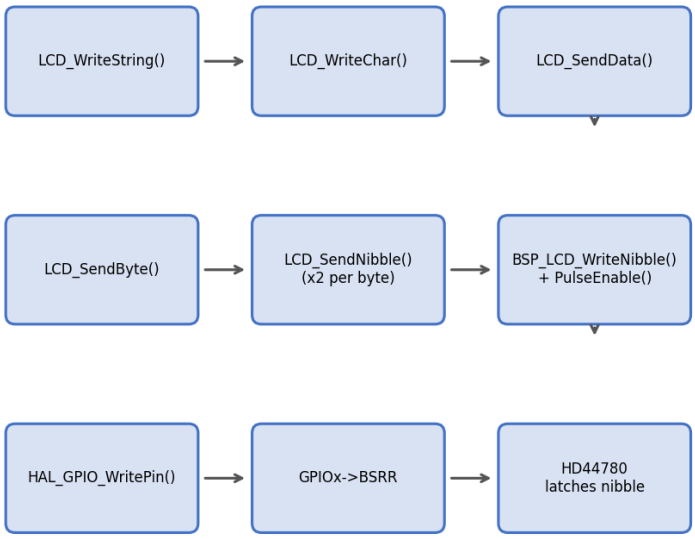

Figure 9.1 — Complete Call Flow: LCD_WriteString() to Hardware

Every character in the string repeats the full chain: LCD_WriteChar -> LCD_SendData -> LCD_SendByte -> two calls to LCD_SendNibble -> BSP_LCD_WriteNibble + BSP_LCD_PulseEnable -> HAL_GPIO_WritePin -> GPIOx->BSRR -> physical pin transition -> HD44780 latches the nibble on the EN falling edge.

Chapter 10 — Initialization Sequence

LCD_Init() must bring the controller from an unknown post-power-up state to a fully configured, known state. Every step below exists because either the HD44780 datasheet mandates it, or because real-world LCD modules (particularly low-cost clones) are not reliable enough to skip it.

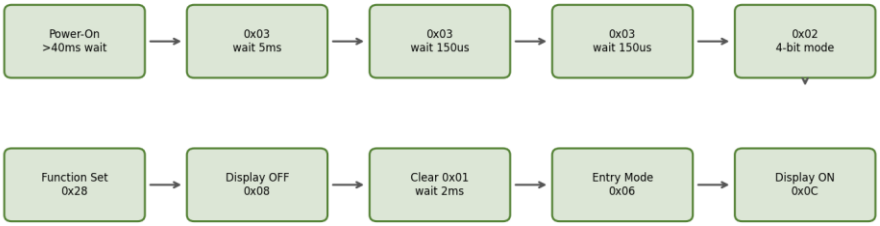

Figure 10.1 — Initialization Sequence Flowchart

Step	Action	Why
1	Wait >= 40ms after VDD reaches operating level	Internal power-on reset circuit needs supply to stabilize; 50ms used in this driver for margin
2	Send nibble 0x03 (as if 8-bit Function Set), wait 5ms	Forces the controller out of any unknown mode into a known 8-bit-interface intermediate state; the datasheet's own recommended re-sync sequence
3	Send nibble 0x03 again, wait 150us	Second re-sync; the controller may have been mid-instruction from a partial/garbled power-up transfer
4	Send nibble 0x03 a third time, wait 150us	Third re-sync — this triple-0x03 pattern is the datasheet-documented recovery sequence and is not arbitrary repetition
5	Send nibble 0x02	Switches the controller from 8-bit to 4-bit interface mode
6	Function Set 0x28 (via LCD_SendCommand, full byte)	Confirms 4-bit mode, 2-line display, 5x8 font — must be

Step	Action	Why
		sent as a full byte now that 4-bit framing is active
7	Display OFF 0x08	Prevents visual glitches on screen while Clear/Entry Mode are applied
8	Clear Display 0x01, wait 2ms	Establishes a known blank DDRAM state
9	Entry Mode Set 0x06	Address counter increments after each write, no display shift — standard left-to-right text entry
10	Display ON 0x0C	Makes the display visible with cursor and blink both off

WARNING: Why steps 2-4 use LCD_SendNibble(), not LCD_SendCommand()

Before step 5, the controller is not yet in 4-bit mode, so it is still expecting a full 8-bit transfer per nibble sent (the upper nibble alone is interpreted as if it were the full command with the lower nibble undefined/ignored). Calling LCD_SendCommand() here would incorrectly send two nibbles per call and desynchronize the sequence. This is why the private LCD_SendNibble() function is exposed as a controlled exception inside LCD_Init() and only there.

Chapter 11 — Command Macros

Every value below is defined as a named macro in driver_lcd.h so no 'magic number' ever appears in driver_lcd.c. This table gives the bit-level encoding and execution time for each.

Hex	Binary	Macro	Purpose	Typical exec. time
0x01	0000 0001	LCD_CMD_CLEAR_DISPLAY	Clears DDRAM, resets AC to 0, cursor home	~1.52-1.64ms
0x02	0000 0010	LCD_CMD_RETURN_HOME	Resets AC to 0 and undoes any display shift	~1.52-1.64ms
0x04	0000 0100	LCD_CMD_ENTRY_MODE_DEC	Entry mode: AC decrements after write	~37-43us
0x06	0000 0110	LCD_CMD_ENTRY_MODE_INC	Entry mode: AC increments after write	~37-43us
0x08	0000 1000	LCD_CMD_DISPLAY_OFF	Display, cursor, blink all off	~37-43us
0x0C	0000 1100	LCD_CMD_DISPLAY_ON	Display on, cursor off, blink off	~37-43us
0x0E	0000 1110	LCD_CMD_CURSOR_ON	Display on, underline cursor on	~37-43us
0x0F	0000 1111	LCD_CMD_CURSOR_BLINK	Display on, cursor on, blink on	~37-43us
0x20	0010 0000	LCD_CMD_FUNCTION_SET_4BIT_1LINE	4-bit interface, 1 display line, 5x8 font	~37-43us
0x28	0010 1000	LCD_CMD_FUNCTION_SET_4BIT_2LINE	4-bit interface, 2 display lines, 5x8 font	~37-43us
0x30	0011 0000	LCD_CMD_FUNCTION_SET_8BIT_1LINE	8-bit interface, 1 line (not	~37-43us

Hex	Binary	Macro	Purpose	Typical exec. time
			used, 4-bit design)	
0x38	0011 1000	LCD_CMD_FUNCTION_SET_8BIT_2LINE	8-bit interface, 2 lines (not used, 4-bit design)	~37-43us
0x40	0100 0000	LCD_CMD_SET_CGRAM_ADDR	Bits 5-0 select CGRAM address (0-63)	~37-43us
0x80	1000 0000	LCD_CMD_SET_DDRAM_ADDR	Bits 6-0 select DDRAM address; ORed with computed row+col	~37-43us

NOTE: Execution time drives the LCD_Init() delay budget

All 'short' commands complete in well under 100us; only Clear Display and Return Home require the explicit millisecond-scale delay. This is why LCD_Clear() and LCD_Home() call HAL_Delay(2) but LCD_DisplayOn()/Off() do not add any delay at all — the next instruction, whatever it is, will naturally not be sent before the current one completes given typical code execution time, but Clear/Home are the two exceptions large enough that this can't be assumed.

Chapter 12 — DDRAM and CGRAM

12.1 DDRAM Address Map — 16x2

Row	Base address (LCD_ROWS>=2)	Valid column range
0	0x00	0x00 - 0x0F (16 columns)
1	0x40	0x40 - 0x4F (16 columns)

12.2 DDRAM Address Map — 20x4

20x4 modules do not use four contiguous 20-byte blocks; the controller's DDRAM is still organized as two 40-byte physical rows, and rows 2 and 3 are offset continuations of rows 0 and 1 respectively. This driver's LCD_ROW_2_ADDRESS / LCD_ROW_3_ADDRESS macros already encode the correct offsets:

Row	Base address	Valid column range
0	0x00	0x00 - 0x13 (20 columns)
1	0x40	0x40 - 0x53 (20 columns)
2	0x14	0x14 - 0x27 (20 columns)
3	0x54	0x54 - 0x67 (20 columns)

WARNING: 20x4 is not four independent 20-byte rows

Row 2 physically continues from address 0x14, which sits inside the same 40-byte DDRAM bank as row 0, not immediately after row 1. A driver that assumes row bases are evenly spaced (0x00, 0x14, 0x28, 0x3C) will display corrupted text on genuine HD44780 20x4 modules. This driver's LCD_ROW_2_ADDRESS = 0x14 / LCD_ROW_3_ADDRESS = 0x54 macros are correct for the standard controller and must not be 'simplified'.

12.3 Cursor Address Calculation

```
address = LCD_RowAddress[row] + col;
LCD_SendCommand(LCD_CMD_SET_DDRAM_ADDR | address);
```

Example: LCD_SetCursor(2, 5) on a 20x4 display computes address = 0x14 + 5 = 0x19, and sends command 0x80 | 0x19 = 0x99.

12.4 Custom Characters (CGRAM)

Although not used by the current driver_lcd.c, the command set supports defining up to 8 custom glyphs (codes 0x00-0x07). The procedure: send LCD_CMD_SET_CGRAM_ADDR | (index*8), then send 8 data bytes, each representing one pixel row (5 bits significant) of the 5x8 character. After programming, switch back to DDRAM addressing with LCD_CMD_SET_DDRAM_ADDR before continuing normal text output.

BEST PRACTICE: Future extension point

If the EV charger UI later needs icons (e.g. a charging-plug glyph, a fault symbol), CGRAM programming should be added as new public APIs (e.g. LCD_DefineChar()) in driver_lcd.c rather than bypassing the driver from the UI layer.

Chapter 13 — Complete Call Flow

This chapter traces every public API end-to-end, from the call site down to the physical LCD pins, to make the full cost of each API call explicit.

13.1 LCD_WriteString("OK") — full trace

```
LCD_WriteString("OK")
  -> LCD_WriteChar('O')
       -> LCD_SendData(0x4F)
            -> LCD_SendByte(0x4F, RS=1)
                 -> BSP_LCD_SetRS(1)              // RS pin HIGH
                 -> LCD_SendNibble(0x4)            // upper nibble
                      -> BSP_LCD_WriteNibble(0x4)  // D4-D7 = 0100
                      -> BSP_LCD_PulseEnable()      // latch
                 -> LCD_SendNibble(0xF)            // lower nibble
                      -> BSP_LCD_WriteNibble(0xF)  // D4-D7 = 1111
                      -> BSP_LCD_PulseEnable()      // latch
  -> LCD_WriteChar('K')  (repeats the same chain for 0x4B)
```

Total GPIO writes for a two-character string: 2 characters x (1 RS write + 2 nibble writes x 4 pins + 2 enable pulses) — the RS write only happens once per byte, so 2 x (1 + 8 + 2) = 22 individual HAL_GPIO_WritePin() calls.

13.2 LCD_SetCursor(1, 4) — full trace

```
LCD_SetCursor(1, 4)
  -> bounds check: 1 < LCD_ROWS, 4 < LCD_COLUMNS  (pass)
  -> address = LCD_RowAddress[1] + 4 = 0x40 + 4 = 0x44
  -> LCD_SendCommand(0x80 | 0x44) = LCD_SendCommand(0xC4)
       -> LCD_SendByte(0xC4, RS=0)
            -> BSP_LCD_SetRS(0)
            -> LCD_SendNibble(0xC)  -> BSP_LCD_WriteNibble + PulseEnable
            -> LCD_SendNibble(0x4)  -> BSP_LCD_WriteNibble + PulseEnable
```

Chapter 14 — HAL Explanation

14.1 HAL_GPIO_WritePin()

Covered at register level in Section 6.6. Summary: a single atomic write to the port's BSRR register, avoiding any read-modify-write hazard on ODR.

14.2 HAL_Delay()

Blocks the calling context by polling a millisecond tick counter (uwTick) that is incremented by the SysTick interrupt, configured during HAL_Init() to fire every 1ms. HAL_Delay(N) busy-waits (yielding to any higher-priority interrupt) until uwTick has advanced by at least N. This is why HAL_Delay() is unsuitable for true microsecond timing — its resolution is bounded by the SysTick period, 1ms in the default HAL configuration used by this project.

14.3 GPIO Initialization and Output Mode

MX_GPIO_Init() in main.c configures LCD_RS_Pin, LCD_EN_Pin, LCD_D4_Pin..LCD_D7_Pin, and BUZZER_Pin as GPIO_MODE_OUTPUT_PP (push-pull output) at GPIO_SPEED_FREQ_LOW. Push-pull means the pin can actively drive both HIGH and LOW (as opposed to open-drain, which can only actively drive LOW). Low speed is appropriate here because LCD control signals do not need fast edge rates; using a higher speed setting would only increase EMI without functional benefit.

14.4 What HAL Hides From the Developer

- Direct RCC clock-enable register bit positions for each GPIO port (__HAL_RCC_GPIOD_CLK_ENABLE() expands to a masked register write).
- GPIO mode/speed/pull configuration register layout (MODER, OSPEEDR, PUPDR bit-field packing per pin).
- The BSRR atomic set/reset trick, exposed as a simple state parameter.

14.5 Advantages and Disadvantages of HAL

Advantage	Disadvantage
Portable across STM32 families with similar API	Extra function-call overhead vs direct register access
Fewer register-level bugs (masking, offsets handled centrally)	Harder to guarantee cycle-exact timing without escaping to CMSIS/registers directly
Self-documenting call sites (HAL_GPIO_WritePin vs raw register writes)	Larger code size than hand-rolled register access

NOTE: Why this design accepts HAL overhead for LCD I/O

LCD refresh is not a hard real-time path in this EV charger firmware; the extra cycles from HAL abstraction are irrelevant compared to the millisecond-scale delays already present in LCD_Init() and LCD_Clear(). Register-level access is reserved for genuinely timing-critical paths such as ADC/DMA sampling and CP state timing (IEC 61851), not the LCD.

Chapter 15 — Timing

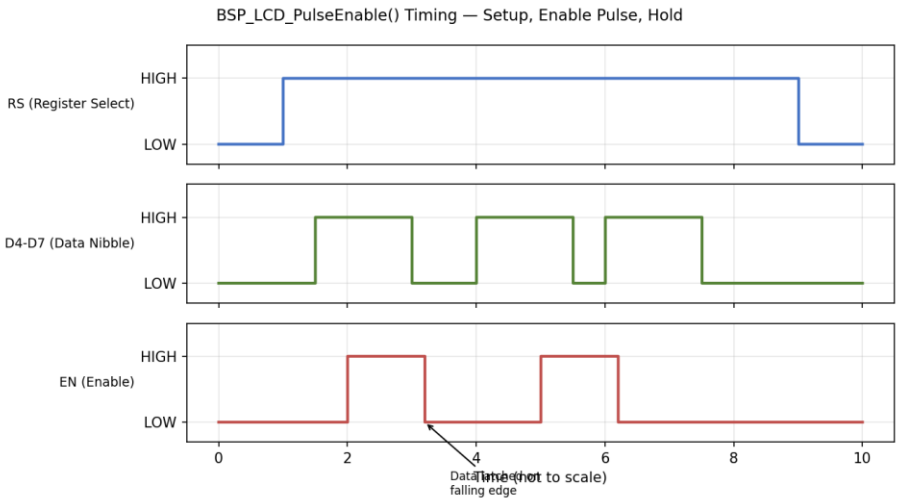

Figure 15.1 — Enable Pulse Timing Diagram

Parameter	Typical HD44780 minimum	This driver's behavior
EN pulse width (high)	~450ns	~1us via BSP_LCD_DelayUs(1) — actually ~1ms due to Section 6.5's HAL_Delay(1) implementation, well above minimum
RS/data setup time before EN rises	~60ns	Guaranteed by instruction sequencing; BSP_LCD_SetRS()/WriteNibble() complete before PulseEnable() begins
Data hold time after EN falls	~10ns	Guaranteed; next nibble write does not begin until PulseEnable() returns
Command execution (short)	~37-43us	Bounded below by ~1ms delay chain between calls; always compliant
Clear Display / Return Home	~1.52-1.64ms	Explicit HAL_Delay(2) in LCD_Clear()/LCD_Home()

Setup Time: the interval RS and the data nibble must be stable before EN rises. Hold Time: the interval they must remain stable after EN falls. Both are satisfied structurally in this driver because BSP_LCD_SetRS() and BSP_LCD_WriteNibble() always complete (and return) before BSP_LCD_PulseEnable() is invoked — there is no possibility of RS or data changing mid-pulse given the strictly sequential, non-interrupt-driven call chain.

> **NOTE: Character write timing budget**
>
> Writing one character costs two enable pulses. At the current ~1ms-per-delay-call implementation of BSP_LCD_DelayUs(), the practical throughput is on the order of a few characters per millisecond-scale budget rather than the microsecond-scale throughput the HD44780 itself is capable of. For a status LCD refreshed a few times per second this is not a functional problem, but it should be understood before this driver is reused for anything requiring faster refresh (e.g. animated bar graphs).

Chapter 16 — Generic Driver (16x2 / 20x4 Support)

```
#define LCD_ROWS        (4U)   // 4 for 20x4 or 16x4, 2 for 16x2
#define LCD_COLUMNS     (16U)  // 20 for 20x4, 16 for 16x2/16x4

#if (LCD_ROWS == 1U)
  #define LCD_CMD_FUNCTION_SET LCD_CMD_FUNCTION_SET_4BIT_1LINE
#else
  #define LCD_CMD_FUNCTION_SET LCD_CMD_FUNCTION_SET_4BIT_2LINE
#endif
```

LCD_ROWS and LCD_COLUMNS are the only two values that change between supported display geometries. Every other line of driver_lcd.c is geometry-agnostic:

- LCD_SetCursor() bounds-checks against these macros rather than hard-coded limits.
- The row-base-address lookup table (Chapter 12) is fixed by the HD44780 controller itself, not by display size — the same table serves both 16x2 and 20x4, and rows beyond LCD_ROWS are simply never indexed.
- LCD_CMD_FUNCTION_SET is resolved at compile time to select 1-line vs 2-line controller mode (HD44780 has no native '4-line' mode; 4-line displays are 2-line controllers with the DDRAM addressing trick from Section 12.2).

BEST PRACTICE: Changing display geometry

To retarget this driver at a different physical LCD, change only LCD_ROWS / LCD_COLUMNS in driver_lcd.h (and confirm LCD_ROW_2/3_ADDRESS match the target module's datasheet if it is a non-standard 4-line variant). No other file requires modification.

Chapter 17 — Debugging

Symptom	Likely cause	Fix
Blank display, backlight on	Contrast (VEE/V0) not set, or LCD_Init() never called	Adjust contrast potentiometer; verify LCD_Init() executes before other LCD_* calls
Only first line ever updates	Row base addresses wrong, or LCD_ROWS misconfigured for a 20x4 module	Verify LCD_ROW_2_ADDRESS=0x14 / LCD_ROW_3_ADDRESS=0x54; confirm LCD_ROWS matches physical hardware
Random / garbled characters	Nibble order reversed, or one data pin wired to wrong GPIO port	Confirm upper-nibble-first in LCD_SendByte(); verify each BSP_LCD_WriteNibble() pin/port pairing (Section 6.3)
Nothing works, boxes only, no init effect	Init sequence interrupted or timing violated (e.g. missing delay after 0x03 x3)	Verify delays in Chapter 10 are actually executing (not optimized out); check BSP_LCD_DelayUs() timing
Works after reset but not after warm power-cycle	Relying on internal HD44780 power-on-reset instead of the explicit 0x03x3/0x02 recovery sequence	Confirm LCD_Init() always performs the full software reset, not a shortcut path
Very low or no contrast even after potentiometer adjustment	Wrong VEE polarity/range or module needs negative bias	Check module datasheet — some modules need a negative V0 relative to VSS
Text present but cursor position wrong after LCD_SetCursor()	Off-by-one in row/col, or DDRAM address table mismatched to physical module	Re-derive address using Section 12.3 formula and compare against measured output

17.1 Oscilloscope Debugging

- Probe EN and one data line together; confirm data is stable before EN's rising edge and remains stable past the falling edge (setup/hold, Chapter 15).
- Measure EN pulse width; compare against the module's datasheet minimum (typically 450ns-1us depending on manufacturer).
- Probe RS transitions relative to EN pulses to confirm command/data bytes are being framed as intended.

17.2 Logic Analyzer Debugging

- Capture RS, EN, D4-D7 simultaneously; most logic analyzer LCD/HD44780 protocol decoders can reconstruct the transmitted byte stream directly from a 4-bit capture.
- Compare the decoded byte stream against the expected initialization sequence in Chapter 10 to catch a missed or duplicated step.

> **COMMON MISTAKE: Debugging with only a multimeter**
>
> A multimeter cannot resolve the EN pulse timing or nibble ordering that cause the vast majority of LCD bring-up issues; it can only confirm DC logic levels are reaching the correct pins at all, which is a necessary but far from sufficient check.

Chapter 18 — EV Charger Integration

This LCD driver subsystem exists to serve exactly one caller category: the UI layer, which in turn is the only component permitted to be called by the EVSE charger state machine (HSM) when it needs to communicate status to the user.

```
EVSE HSM (IEC 61851-1 states A-F, fault handling)
        |
        v
UI  (maps HSM state -> screen text/layout)
        |
        v
LCD Driver  (driver_lcd.c/h)
        |
        v
BSP  (bsp_lcd.c/h)
        |
        v
STM32 HAL
        |
        v
GPIO -> HD44780 LCD
```

18.1 Why the HSM Must Never Call LCD APIs Directly

- Determinism: the HSM's state transitions are safety-relevant (CP state sampling, ventilation requirements, GFCI/RCD fault response per IEC 61851-1). Any blocking delay inside LCD_Clear()/LCD_Home() (2ms) or the character-write chain must never be allowed inside a safety-timing-critical HSM tick.
- Separation of concerns: the HSM should express intent ('charging in progress', 'fault: ground fault detected') without knowing display geometry, cursor coordinates, or LCD command timing.
- Testability: the HSM can be unit-tested against a mocked UI interface without any LCD or GPIO dependency at all.

WARNING: Blocking delays inside a safety-relevant control loop

LCD_Clear() alone blocks for 2ms via HAL_Delay(). If the EVSE HSM's control tick period is itself only a few milliseconds (as is common for CP state sampling), calling any LCD API directly from the HSM tick can measurably distort control timing. Routing all display requests through the UI layer, decoupled from the HSM tick (e.g. via a queue or a lower-priority task/loop), avoids this entirely.

18.2 Recommended Integration Pattern

- HSM posts high-level status events (enum-based) to the UI layer; it never touches LCD_* or BSP_* symbols.
- UI layer owns a simple state-to-text mapping table and issues LCD_SetCursor()/LCD_WriteString() calls only from a non-safety-critical context (e.g. main loop background task, not inside CP-state ISR handling).
- If the HSM tick rate is tight enough that even the UI layer's LCD writes risk interference, defer LCD updates to a lower-priority cooperative task and rate-limit refreshes (e.g. 4-10 Hz is more than sufficient for human-readable status text).

References

- Hitachi HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver — datasheet.
- STMicroelectronics, STM32F407xx Reference Manual (RM0090).
- STMicroelectronics, STM32F4 HAL Driver User Manual (UM1725).
- IEC 61851-1:2017, Electric vehicle conductive charging system — Part 1: General requirements.
- Internal project source: main.c, driver_lcd.c/h, bsp_lcd.c/h (this repository).

Appendix A — Glossary

Term	Definition
DDRAM	Display Data RAM — holds character codes for currently visible display positions
CGRAM	Character Generator RAM — up to 8 user-defined custom character glyphs
CGROM	Character Generator ROM — fixed built-in font table
AC	Address Counter — internal HD44780 register tracking current DDRAM/CGRAM address
RS	Register Select — chooses Instruction Register (0) or Data Register (1)
EN	Enable — the strobe line whose falling edge latches data into the controller
BSP	Board Support Package — the layer isolating physical pin/port assignment
HSM	Hierarchical/Hardware State Machine — here, the EVSE charger control state machine
CP	Control Pilot — the IEC 61851-1 signal used for EV/EVSE state negotiation